

Pixel Camera Pro

A smooth pixel-perfect 2D camera system for Unity URP with per-layer resolutions and built-in parallax. Cinemachine compatible.

Contents

1. Overview	2
2. Setup Walkthrough	2
2.1. Create pixel camera	2
2.2. Add Pixelation Layers	3
2.3. Set the camera's zoom	4
2.4. Set up Cinemachine Camera	5
2.5. Set up Parallax	6
2.6. Snap your sprite to the grid	7
2.6.1. Static pixel art	7
2.6.2. Sprites that move at runtime	7
2.6.3. Layers for camera followed pixel sprites	8
2.7. Verify Your Setup	9
3. Core Concepts	10
3.1. Pixelation Layer	10
3.2. Letterbox	10
3.3. The = Button	10
3.4. Pixels Per Unit (PPU)	10
3.5. Camera Scale & Minimum Camera Scale	11
3.6. Snap to Pixel Grid	11
4. Clean Up Unused Quad Display Layers	11
5. Scripting API	12
6. Troubleshooting / FAQ	13
6.1. Lighting flickers as the camera moves.	13
6.2. Pixel-art sprites look unstable while the camera moves.	13
6.2.1. Pixelated sprite's shape is not consistent.	13
6.2.2. Camera-followed sprites shimmer as camera moves.	13
6.3. A layer's Pixel Size warning won't go away.	13
6.4. Cinemachine fights my camera's zoom.	13
6.5. Black bars appear at the top and bottom of your screen.	13
6.6. "No empty Layer slot" warning in the console.	13
7. Supports	14

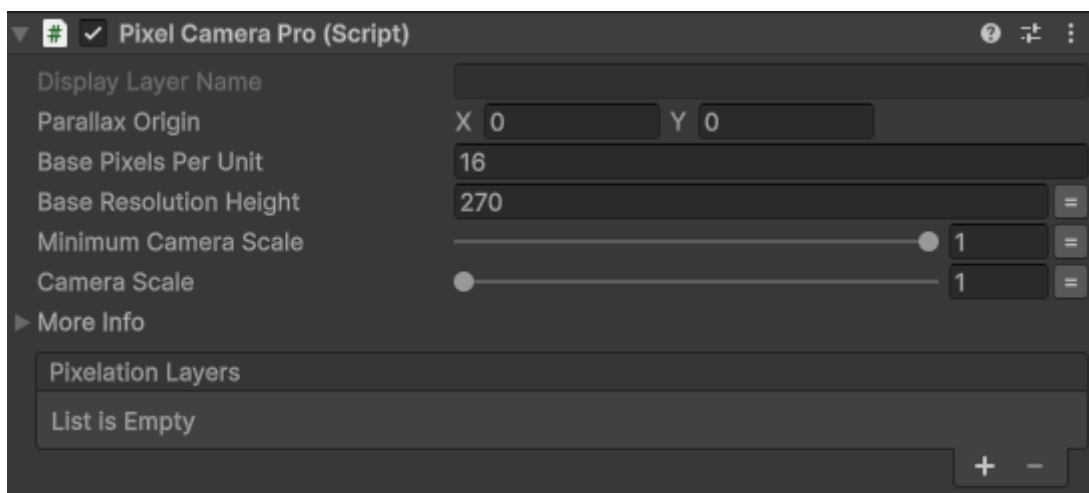
1. Overview

Pixel Camera Pro is a *pixel-perfect* camera system designed for 2D pixel art games. It delivers a smooth, *jitter-free* camera movement while allowing you to render different parts of your scenes at different resolutions. Each layer stays *pixel-perfect* on its own, making *smooth parallax* possible.

2. Setup Walkthrough

2.1. Create pixel camera

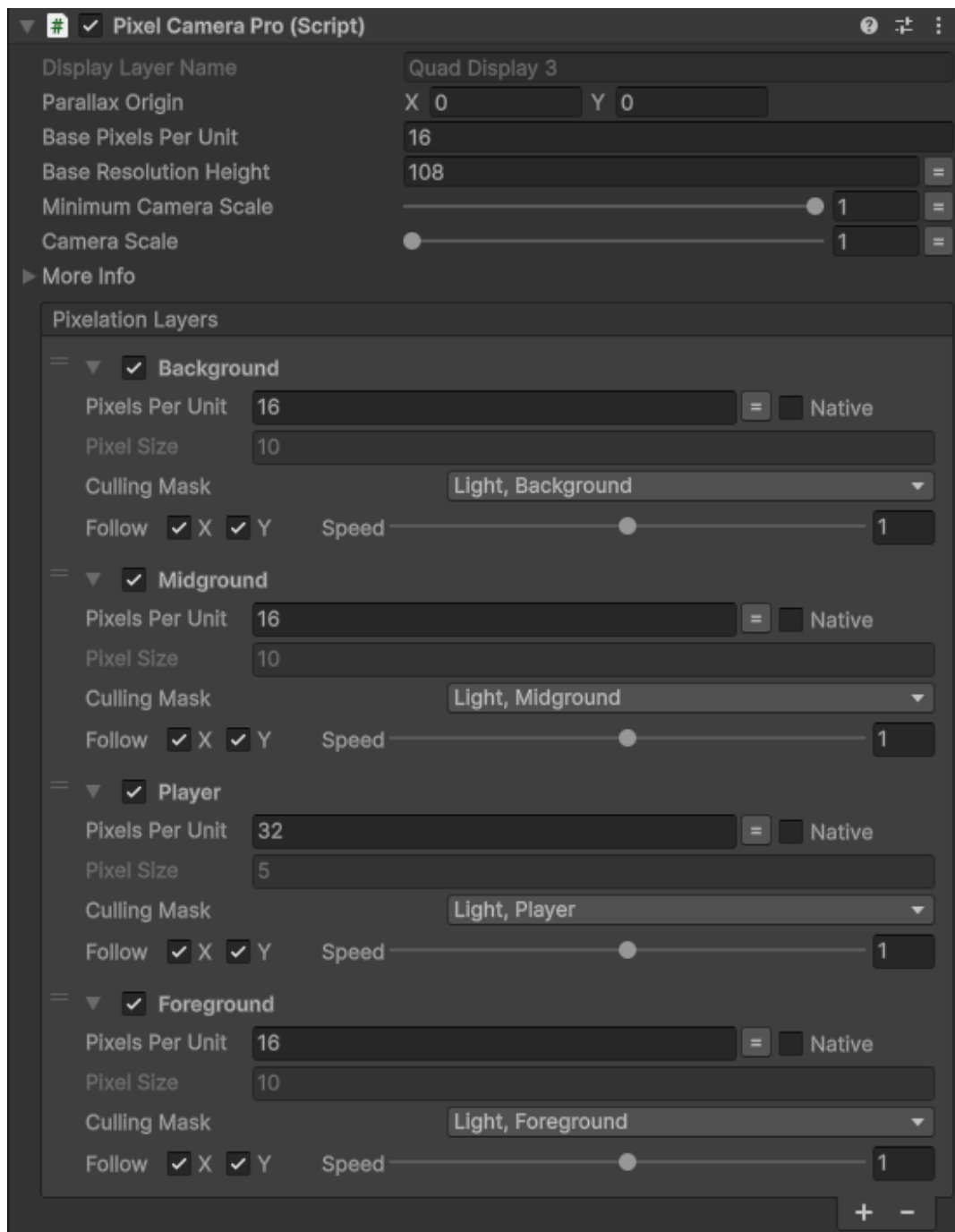
1. Add a camera to your scene, then add the **Pixel Camera Pro** component to the GameObject.
2. Set **Base Pixels Per Unit** to whatever PPU most of your sprites are imported at.
3. Set **Base Resolution Height** to the screen height your pixel art is designed for. Click the **=** button and pick from the recommended list. Every value there divides your current window height evenly, so it fills the screen with no [letterboxing](#) . Typing your own value works too, but the list guarantees this.



Note: Pixel Camera behaves like a normal camera. Supports camera stack and cinemachine.

2.2. Add Pixelation Layers

1. In the Pixelation Layers list at the bottom of the Inspector, click **+**. This creates a pixelation layer. Double-click a layer's name to rename it.
2. Set the layer's **Culling Mask** to the Unity Layer(s) containing the sprites you want to render.
3. Set **Pixels Per Unit** to the PPU you want for this layer. Click the **=** button to display all PPU values that produce square, whole-number pixels at the current window size. Or toggle **Native** instead when you want the layer to render at full screen resolution.
4. Repeat for each additional layer.
5. Layers are rendered from top to bottom in this list. Drag layers to change their render order.
6. You can click the triangle button to collapse a layer.



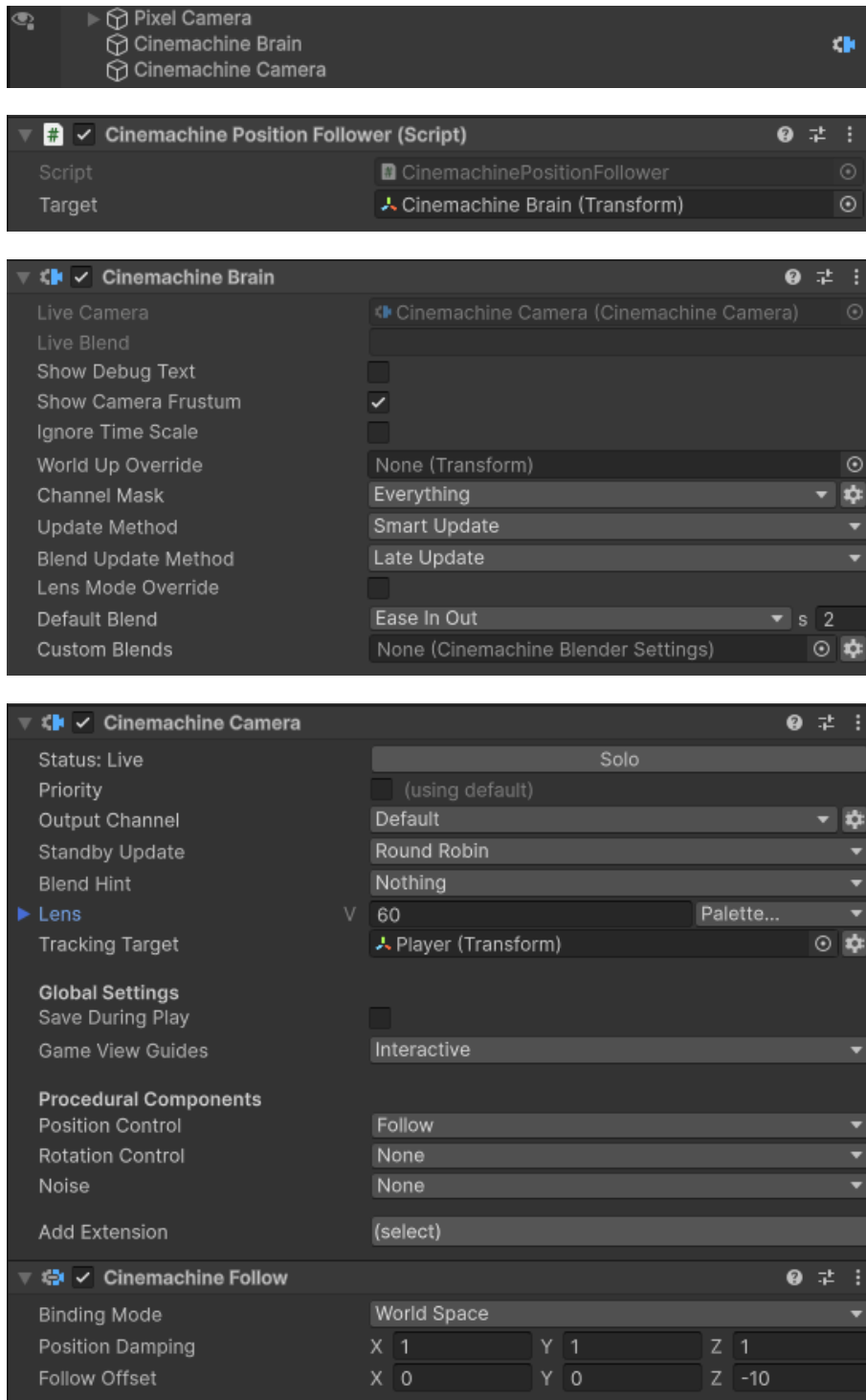
2.3. Set the camera's zoom

1. Set **Minimum Camera Scale** to the most zoomed-out view your game will ever use. Use its `=` button to pick a value every one of your layers stays clean at. It takes each layer's Pixels Per Unit into account and finds a scale that keeps pixels square and aligned to whole numbers across all layers.
2. Set **Camera Scale** to your camera's zoom level. 1 is the normal view, and higher numbers zoom in. This is also the value you'd change at runtime to zoom the camera in and out during play. Its own `=` button lists whole zoom levels that keep every layer clean, the same way Minimum Camera Scale's does. Any whole multiple of Minimum Camera Scale stays clean too, even ones the list doesn't show.

You can adjust Camera Scale smoothly at runtime (for example, by lerping toward `BaMin` a target value) to create smooth zoom transitions. Pixels may look slightly uneven mid-transition. This is expected, and usually unnoticeable when the camera is changing quickly. Just make sure that once the zoom finishes, Camera Scale settles on a value that doesn't trigger a Pixel Size warning on any layer. A whole-number multiple of Minimum Camera Scale or an integer value (such as 1, 2, 3, etc.) is always guaranteed to work, but depending on your layers' Pixels Per Unit, other values may work too.

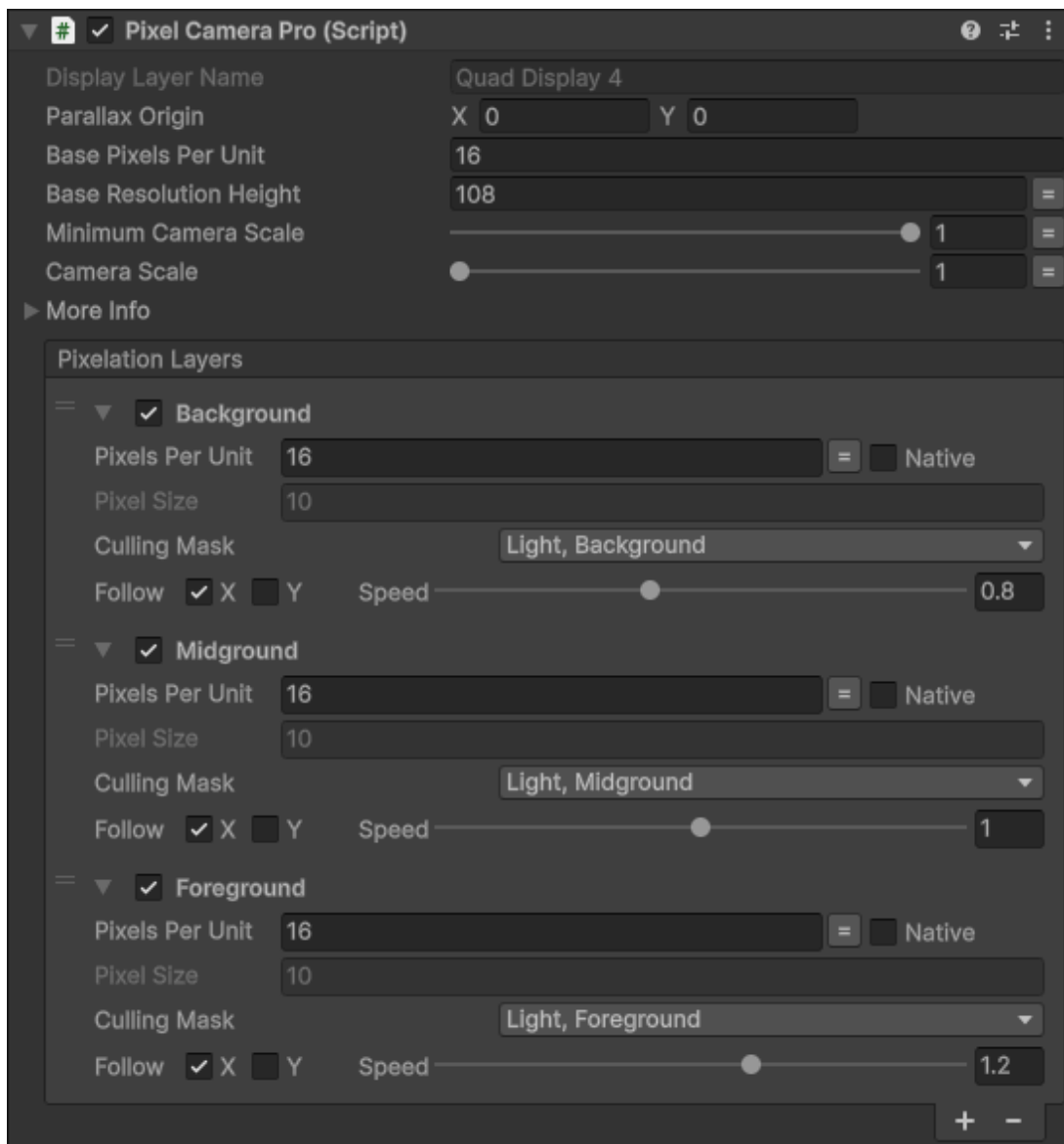
2.4. Set up Cinemachine Camera

1. Skip this part if you're not using Cinemachine.
2. Set up your Cinemachine Camera as usual, but place it on a separate empty GameObject from your pixel camera .
3. Add a **Cinemachine Position Follower** component to your pixel camera, and set its **Target** field to that GameObject.
4. Control zoom through **Camera Scale** directly instead of your Cinemachine Camera's Lens.



2.5. Set up Parallax

1. Skip this part if your game doesn't have parallax.
2. Add a Pixelation Layer for each parallax layer you want to move at its own speed.
3. **Speed** controls how fast a layer moves relative to the camera. Below **1**, the layer moves slower than the camera and appears farther away. Above **1**, it moves faster and appears closer. at **1**, it tracks the camera normally.
4. Use **Follow X** and **Follow Y** to lock a layer to a single axis.
5. Set **Parallax Origin** (near the top of the Inspector) to wherever your parallax art should line up with the rest of the scene, usually your camera's starting position. That's the point every layer's parallax is measured from.



2.6. Snap your sprite to the grid

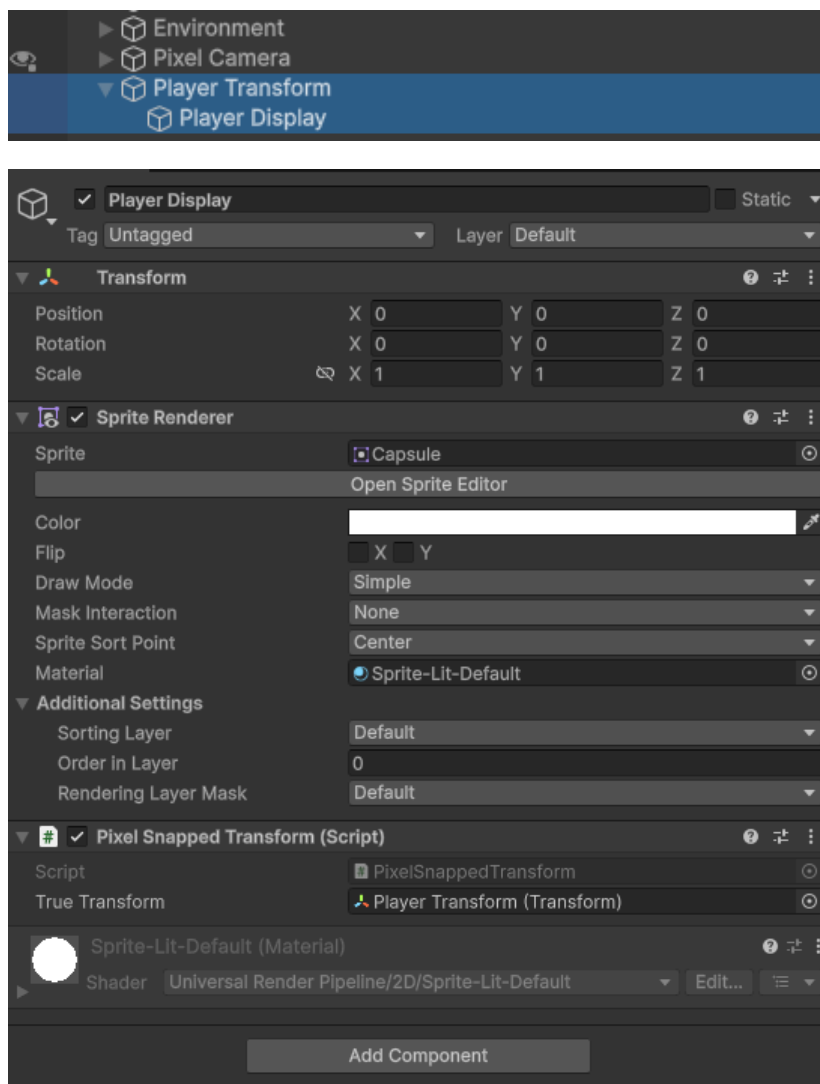
2.6.1. Static pixel art

1. Select the pixel-art sprites already placed in your scene (or their common parent), right-click, and choose **Pixel Camera Pro > Snap to Pixel Grid**.
2. Do this before entering Play Mode. An un-snapped pixel-art sprite will look unstable as the camera moves.

2.6.2. Sprites that move at runtime

The step above only fixes a position once, in the editor. It can't help a character or a script-driven object that gets a new position every frame. For those, instead:

1. Create a child GameObject under the moving object, and move its **SpriteRenderer** (and any **Animator**) onto that child.
2. Add a **Pixel Snapped Transform** component to the child, and set its **True Transform** field to the moving object, its own parent.
3. The child now snaps to the grid every frame, following its parent, while the parent itself keeps moving exactly as your own script or physics expects.



2.6.3. Layers for camera followed pixel sprites

A layer that snaps to the pixel grid keeps pixel art crisp and stable, but it makes camera followed pixel sprites appear to shimmer. A simple solution is to render the layer at the sprite's PPU by default and temporarily use native resolution only while the object is moving above a certain speed. Once its movement slows or stops, disable Native to restore smooth, jitter-free rendering.

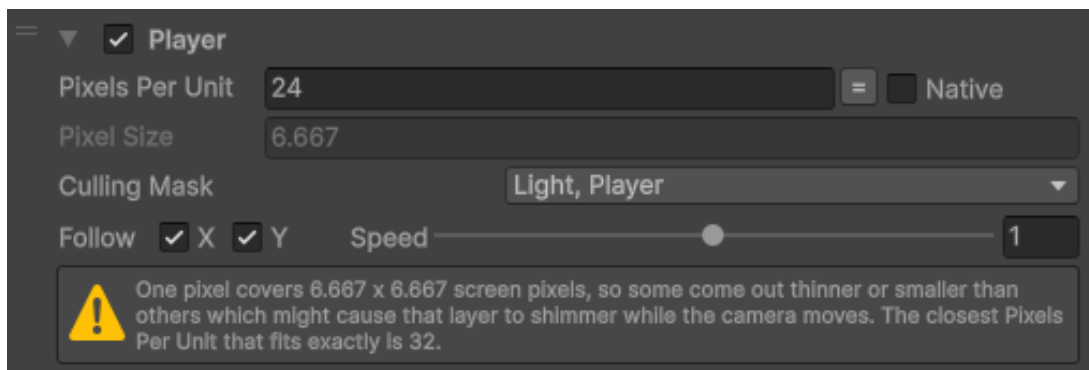
1. Give those objects their own Pixelation Layer, if they didn't have one already.
2. Control that layer's **Native** setting from a script. Keep `IsNativeResolution` disabled while the object is stationary or moving slowly, and enable it once its speed exceeds a chosen threshold.

```
Frequently called 1 usage greedjesse *  
private void HandleResolution()  
{  
    var speed :float = Vector2.SqrMagnitude(rigidbody2D.linearVelocity);  
    pixelCamera.Layers[0].IsNativeResolution = speed > threshold;  
}
```

2.7. Verify Your Setup

That's it, your scene should now be pixelated!

Every Pixelation Layer shows a live **Pixel Size** readout, with a warning if that layer's pixels aren't coming out whole and square. That warning usually means this layer's Pixels Per Unit doesn't fit your current window size cleanly, which can shimmer as the camera moves. It can also happen in play mode, if **Camera Scale** isn't currently a whole multiple of Minimum Camera Scale or an integer (for example, mid-zoom from your own script). If you see a warning, try adjusting **Base Pixels Per Unit**, or use the  button next to **Base Resolution Height**, **Minimum Camera Scale**, **Camera Scale**, or the layer's own **Pixels Per Unit** to choose a recommended value.



Important: It's highly recommended to use the  button whenever possible. It's specifically designed to find values that produce square, whole-number pixels with your current settings.

3. Core Concepts

3.1. Pixelation Layer

Each Pixelation Layer renders part of your scene with its own camera and its own resolution, then stacks that result onto the display camera. Because layers are independent (Each pixel perfect on its own), each one can have its own Pixels Per Unit.

3.2. Letterbox

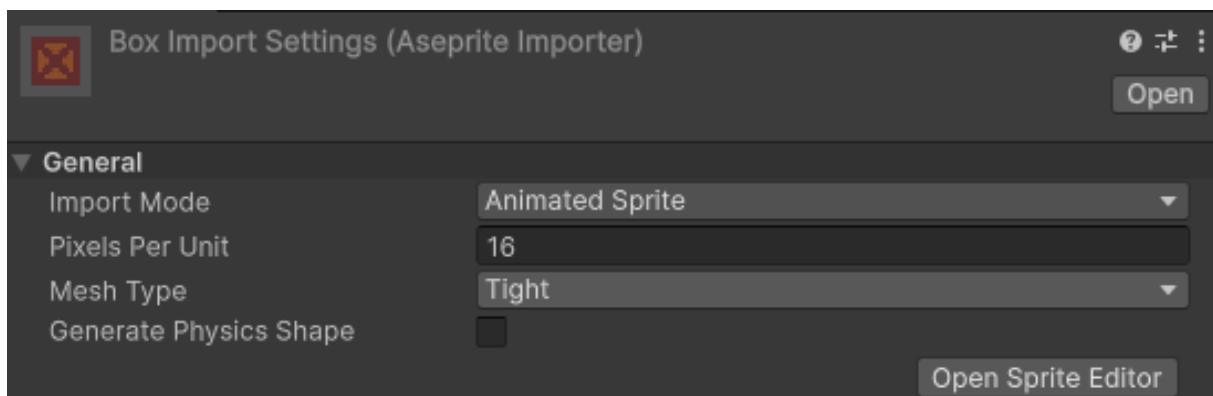
Letterbox is the black bars shown at the top and bottom of the screen if the screen height isn't an exact multiple of **Base Resolution Height**. That's because content can only scale up in whole numbers to keep every pixel a clean, uniform size. Letterbox bars paint that space black instead of stretching content unevenly to cover it. A value from **Base Resolution Height's**  button avoids this for your current window size.

3.3. The Button

Every  button next to a numeric field opens a menu of recommended values, though what that recommendation actually checks differs by field. On **Minimum Camera Scale**, **Camera Scale**, and a layer's own **Pixels Per Unit**, every value listed is checked against your current layers and guaranteed to keep them clean. On **Base Resolution Height**, it only guarantees no letterboxing and an even Base Pixel Size; whether a specific layer's own Pixel Size also comes out clean still depends on that layer's PPU.

3.4. Pixels Per Unit (PPU)

Sets the resolution of a Pixelation Layer. It should match the PPU your sprites are imported at; a mismatch might cause your pixelated sprites to jitter while camera moves. Non-pixel-art sprites can use any PPU you like.



3.5. Camera Scale & Minimum Camera Scale

Camera Scale is the camera's live zoom (1 is normal, above 1 zooms in). **Minimum Camera Scale** is the most zoomed-out value you'll ever set Camera Scale to.

Set Minimum Camera Scale to the smallest headroom you'll actually need, not the smallest number possible: the lower it is, the more world a layer pre-renders.

3.6. Snap to Pixel Grid

Pixel Camera Pro > Snap to Pixel Grid and **Pixel Snapped Transform** work the same way. Each looks at the sprite's own Unity Layer, finds the Pixelation Layer whose Culling Mask covers it, and uses that layer's PPU to snap the sprite to the grid. Because of this, any Unity Layer your snapped sprites live on must be covered by exactly one Pixelation Layer. Layers you never snap, like lighting, can be covered by several with no issue.

4. Clean Up Unused Quad Display Layers

Each Pixel Camera Pro claims its own Unity Layer slot for its Quad Display objects, so multiple cameras in the same scene don't render into each other's display. This runs automatically whenever your scene hierarchy changes, freeing any slot no live camera is using anymore, so deleting a camera frees its Layer on its own. **Tools > Pixel Camera Pro > Clean Up Unused Quad Display Layers** does the same cleanup.

5. Scripting API

Get the Pixel Camera Pro component and set its properties directly,

for example `GetComponent<PixelCameraPro>().CameraScale = 2f; .`

A layer's own properties live on its Pixelation Layer Camera component instead, reachable through

`Layers`, for example `pixelCamera.Layers[0].Speed = 0.5f; .`

Property	Component	Notes
<code>CameraScale</code>	Pixel Camera Pro	Clamped between <code>MinimumCameraScale</code> and <code>5</code> .
<code>ParallaxOrigin</code>	Pixel Camera Pro	Parallax reference point.
<code>BasePixelsPerUnit</code>	Pixel Camera Pro	Clamped to a minimum of <code>1</code> .
<code>BaseResolutionHeight</code>	Pixel Camera Pro	Clamped to a minimum of <code>1</code> .
<code>MinimumCameraScale</code>	Pixel Camera Pro	Snapped to the nearest <code>0.1</code> , clamped between <code>0.5</code> and <code>1</code> .
<code>Speed</code>	Pixelation Layer Camera	Per-layer parallax speed.
<code>FollowX</code>	Pixelation Layer Camera	Locks parallax to the X axis.
<code>FollowY</code>	Pixelation Layer Camera	Locks parallax to the Y axis.
<code>PixelsPerUnit</code>	Pixelation Layer Camera	Clamped to a minimum of <code>1</code> .
<code>IsNativeResolution</code>	Pixelation Layer Camera	Switches between pixelated and native, full-resolution rendering.

A layer's Culling Mask, enabled state, and name are just plain Unity `Camera` and `GameObject` members, not anything Pixel Camera Pro adds itself, so `GetComponent<Camera>().cullingMask` and `gameObject.SetActive( ... )` work exactly like they would anywhere else. You can't add or remove a layer at runtime, creating one relies on Editor-only tooling.

6. Troubleshooting / FAQ

6.1. Lighting flickers as the camera moves.

Please make sure the `Light Render Texture Scale` is set to 1 in your Renderer Data.



6.2. Pixel-art sprites look unstable while the camera moves.

6.2.1. Pixelated sprite's shape is not consistent.

This usually means the sprite's own position isn't snapped to the pixel grid yet. See [Snap your sprite to the grid](#) in the walkthrough. If the sprite is already snapped, check that layer's **Pixel Size** readout instead. An uneven value causes the sprites to shimmer too.

6.2.2. Camera-followed sprites shimmer as camera moves.

If it's specifically for objects followed by camera, this is expected on a layer whose camera snaps to the pixel grid. See [Layers for camera followed pixel sprites](#) for the recommended setup.

6.3. A layer's Pixel Size warning won't go away.

Try to adjust the config using [the ≡ button](#). It's specifically designed to find values that produce square, whole-number pixels with your current settings.

6.4. Cinemachine fights my camera's zoom.

See [Cinemachine](#). Don't put a Cinemachine Brain on the same camera as Pixel Camera Pro, use **Cinemachine Position Follower** to track a separate proxy instead.

6.5. Black bars appear at the top and bottom of your screen.

See [Letterbox](#) under Concepts; the `=` button next to Base Resolution Height avoids this for your current window size.

6.6. "No empty Layer slot" warning in the console.

Unity only gives you 32 Layers total, and Pixel Camera Pro needs one per camera. Free one up in **Edit > Project Settings > Tags and Layers**, or use **Tools > Pixel Camera Pro > Clean Up Unused Quad Display Layers** first to cleanup unused Quad Display layers.

7. Supports

- Discord: <https://discord.gg/9jVjbSnpCH>
- Gmail: greed.jesse.business@gmail.com